

Submitted to the Department of Electrical Engineering and
Computer Science
in partial fulfillment of the requirements for the
Master of Engineering in Electrical Engineering
at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

February 1999

© Shane Michael Swenson, MIT

The author hereby grants to MIT the right to
reproduce and distribute publicly paper and electronic
copies of this thesis in whole or in part.

Author
Department of Electrical Engineering

Certified by.....

Accepted by.....

Chairman, Department Committee

Abstract

Instruction scheduling on software exposed and
formed in both time and space. The complexity
regions dictates that the space-time scheduling
nately, the interaction of phases presents a ph

In this thesis, the structure of program sched
ing regions are shown to have varying charact
simple algorithm to cover. A new scheduling
this diversity and minimize the phase orderi
taining exact mappings of instructions to tim
scheduler maintains probabilities for different
space resources. Second, a set of small schedu
the probabilistic assignments many times in
ordering.

A simple spatial instruction scheduler for
is implemented and shown to outperform exi
erage.

Thesis Supervisor: Saman Amarasinghe
Title: Associate Professor

1 Introduction

- 1.1 Software-Exposed Architectures
- 1.2 Space-Time Instruction Scheduling
- 1.3 Phase Ordering Problem
- 1.4 Combined-Phase Probabilistic Scheduling
- 1.5 Organization

2 Compiling for the Raw Architecture

- 2.1 Raw Architecture
 - 2.1.1 Processing Unit
 - 2.1.2 Communication Network
 - 2.1.3 Distributed Memory
- 2.2 Memory Bank Disambiguation
 - 2.2.1 Equivalence Class Unification
 - 2.2.2 Loop Unrolling and Alignment
- 2.3 Space-Time Scheduler

3 Empirical Analysis of Program Characteristics

- 3.1 Benchmark Suite
- 3.2 Visualizing Scheduling Regions
 - 3.2.1 Choice of Scheduling Regions
 - 3.2.2 Dense Matrix Applications
 - 3.2.3 Multimedia Applications

4 Combined-Phase Probabilistic Sched

- 4.1 Motivation
- 4.2 Detailed Description
 - 4.2.1 Probabilistic Schedule
 - 4.2.2 Local Heuristics
 - 4.2.3 Application of Local Heuristics
- 4.3 Example

5 Algorithm

- 5.1 Probabilistic Schedule Representation
- 5.2 Heuristics Implemented
 - 5.2.1 `do_preplaced_nodes`
 - 5.2.2 `balance_tile_load`
 - 5.2.3 `round_robin_tiles`
 - 5.2.4 `strengthen_max_tiles`
 - 5.2.5 `find_parallelism`
 - 5.2.6 `examine_neighbors`
 - 5.2.7 `highlight_critical_path`
- 5.3 Driver Implementation

6 Implementation and Results

- 6.1 Implementation
- 6.2 Methodology
- 6.3 Speedup

8 Related Work

8.1 MIMD and VLIW Compilation . .

9 Conclusions

9.1 Summary

9.2 Future Work

9.2.1 Scheduling and Register Allo

9.2.2 Better Heuristics and Tuning

9.2.3 Driver Improvement

A Graphs

1-1	Back-end phase diagram for the MIT
2-1	A Raw Microprocessor
3-1	Shapes of nodes assigned to specific
3-2	Cholesky 8 tiles
3-3	Cholesky 8 tiles (4)
3-4	Life 8 tiles (2)
3-5	Mxm 8 tiles (3)
3-6	Vpenta 8 tiles (2)
3-7	Adpcm 8 tiles
3-8	SHA 2 tiles
3-9	Fpppp 8 tiles
3-10	A scheduling region with a critical p
4-1	Combined-phase probabilistic schedu
4-2	Simple example
6-1	Speedup of benchmarks on Raw with to benchmarks compiled with Mach one tile
A-1	Cholesky 8 tiles
A-2	Cholesky 8 tiles (4)
A-3	Life 8 tiles (2)

2.1	Tile processor latencies
3.1	Benchmark characteristics. Sequential processor code generated by the Machs
6.1	Execution cycle counts for probabilistic partitioning of Raw benchmarks

Introduction

In order to maintain a high rate of performance, computer architects must continue to increase the number of instructions that can be exploiting parallelism in applications. Unfortunately, this is often difficult to perform at run time due to wire delays and scheduling parallelism. Therefore, scalable architectures for the future must give compilers control over not only how many but on which processing element that execute instructions and ordering them to resources and scheduling, is critical to the performance of modern architectures [12] [14]. This thesis explores the space of spatial scheduling and offers a new approach based on weighted scheduling. A spatial scheduling system based on this approach is implemented on the MIT Raw architecture. This system yields a significant improvement over the existing spatial scheduling algorithms.

1.1 Software-Exposed Architecture

In order to maintain an increasing rate of performance, computer designers have developed a number of features to allow programs to exploit instruction level parallelism (ILP) in

maintain the single serial processor interface. The design and exploitation of ILP from hardware to software are referred to as *software-exposed architectures*. The performance on a software-exposed architecture is determined by the compiler, which determines how the hardware resources are used for instruction scheduling and register allocation.

1.2 Space-Time Instruction Scheduling

On a conventional monolithic processor, the instruction scheduling by the compiler is purely temporal; the instruction scheduling is a schedule that minimizes completion time with respect to the distributed nature of the computation and scheduling. In a distributed architecture and their exposure to the compiler, the instruction scheduling problem to one that is both spatial and temporal. The compiler must explicitly schedule ILP across the architecture considering the trade-off between communication and computation. Instructions executed in parallel or in sequence are referred to as *scheduling* [12].

To simplify the space-time instruction scheduling, the task is divided into phases. For example, the general purpose task is divided into the phases shown in Figure 1. The task is partitioned into sets that can be executed in parallel. The task is placed on specific computation units. Finally, the task on each computation unit are scheduled temporally. After the task is scheduled temporally, the task is scheduled spatially.

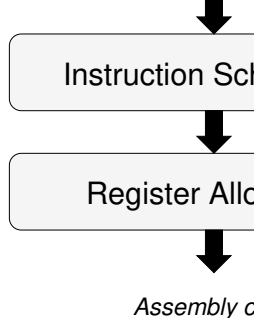


Figure 1-1: Back-end phase diagram.

assigns registers to program variables and temporary code.

1.3 Phase Ordering Problem

This division of the back-end code generation into scheduling and register allocation into multiple phases reduces the computational complexity of the tasks to a tractable level. However, the order in which these phases are performed in each phase often affect the outcomes of the other phases. Furthermore, most decompositions of these tasks into phases is the *phase ordering problem*, meaning that different programs may require different orders of the phases. For example, in some compilers, register allocation is a main performance bottleneck and early register allocation is preferred. In others, instructions with long delays, such as floating-point operations, meaning that priority should be given to temporary code to minimize the effects of those delays.

The phase ordering problem has two primary components:

is necessary to keep the complexity of the intermediate task of building a particular code generation of the other phases as well is identical to the generator.

The second cause of the phase ordering problem is between phases. Generally each compilation phase passes information to the next phase in line. The partitioner takes a list of sets of partitioned instructions, the ins2fns takes a list of sets of instructions to function units, the scheduler takes an ordering of the instructions or a mapping of instructions to function units, the allocator outputs only a mapping variables to registers. A simple interface is that the phases often must make decisions. For example, the partitioner may have to choose between cost function values though one of them will be affected by scheduling or register allocation. The scheduler correctly because it does not know how the instructions will be ordered. Making the decision by completing the code generation of the results quickly becomes intractable if the number of options is not kept extremely small. Similarly, passing information to the allocator allowing them to make the decision also quickly becomes intractable if of equivalent options to be passes will very likely be large.

to represent the assignment of instructions to processors in place of a strict boolean mapping. Thus when the processor has low confidence in, it gives low weight to the decision, leaving it open for future phases to reverse the decision. When the processor makes a decision and assigns a high weight to it, future phases can reverse it by decreasing its weight. The unshared information is passed to all phases to examine the entire probability space. For decisions that did not lead to the final output, the processor is unsure about a decision, it also gives some weight, which can then be examined by future phases.

In the development of this approach, four representative programs were studied to see how much variance in program structure was there. The algorithm to effectively cover all cases, resulting in a combined-phase probabilistic scheduling approach, was applied to the space-time scheduling problem while minimizing the number of processors. Using this approach, a simple spatial scheduling problem was tested, and evaluated on the MIT Raw Machine.

1.5 Organization

The remainder of this thesis is organized as follows. Chapter 2 describes the processor and RAWCC, the sequential compiler, and how they are integrated. In Chapter 3 the characteristics of the problem instances are examined in detail with the

Finally, Chapter 8 describes work related to the
problem for Raw machines and Chapter 9 concludes.

Chapter 2

Compiling for the RAW Architecture

The Raw architecture is a distributed microprocessor architecture beyond the limits of current high performance architectures. The machine is composed of a two dimensional array of tiles. Each tile contains a processing unit, a memory bank, and can execute independent instruction streams, though the architecture is designed to flow and instruction dependencies.

RAWCC is the parallelizing C and FORTRAN compiler implemented on top of the SUIF compiler infrastructure. It has three main components. First, memory disambiguation [1] is performed on the program in order to discover memory access patterns for each particular bank. Then the space-time scheduler [2] is used to assign instructions to tiles, temporally schedule the execution of each tile, and orchestrating any communication between tiles of the computation.

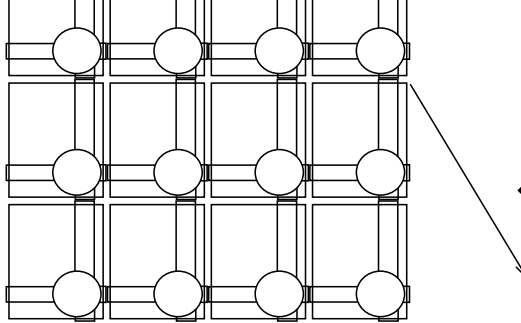


Figure 2-1: A Raw Machine

2.1 Raw Architecture

The MIT Raw architecture [19] is a simple, architecture. A Raw machine, shown in Figure 2-1, consists of identical tiles that execute independent instructions in a simple five-stage RISC pipeline along with a local cache and is interconnected with the other tiles over a network. The tiles are kept simple and small in order to fit on a chip and to facilitate a high clock rate. Each tile is tightly coupled with the processor on each tile to provide a high level of control to the software. Unlike modern superscalar architectures, this architecture is transparent to the software.

2.1.1 Processing Unit

The main processing unit on each tile is a processor that implements the MIPS R4000 instruction set and a small set of instructions. Table 2.1 lists latencies for the instructions.

Integer divide
Floating point add, subtract,
Floating point divide

processing unit. All operations, except for floating point

The network access instructions control register is located in the main processing unit and the switch. The network access instructions are mapped into the register file name space so variables can be accessed as ordinary registers in any instruction that the network access port uses dedicated data pathways that are accessed directly from the network into the opcode. These data pathways are integrated into the processing unit, allowing the processor to access data as soon as they are available, rather than waiting for the next stage of the pipeline. Using this feature, a word of data can be sent to neighboring tiles in only 3 cycles.

2.1.2 Communication Network

The point-to-point communication network consists of a dedicated switch processor on each tile. Each tile is connected to its four neighboring switches via an input/output port. The input/output ports of the machine are used for communicating with the banks.

The network supports both static and dynamic routing. It has the need to compose and route dynamic messages of any size, as small as a single word to be efficiently communicated.

The static network is comprised of the static tiles that connect them to their respective processors. The tiles connect them to the switches on their neighboring tiles. The static network can be programmed, giving the software control over the static network. A word of data in this network takes one clock cycle so the total time to communicate is the Manhattan distance between them plus one cycle to write and one cycle to read it.

The compiler uses this network to communicate. Since the destination tiles are known at compile time, the compiler can schedule instructions due to data dependences since the tile that each instruction is assigned to. Thus most of the communication is done in the static network.

For the static network to route data correctly, each switch along the path must contain instructions to route the data to the switch on the route. This low overhead approach reduces the overhead and increases available bandwidth by eliminating the need to decode a message header for each message. The static network routes means that routing channels can be set up for data destined for them.

In order to assist the static network in routing data, the network implements near-neighbor flow control. Each tile has read and write ports. A write instruction writes to a port if space is available, and a read instruction reads from a port if there is data to be read on the requested port. This eliminates the need for a global buffer.

and I/O events, over that of an architecture of program counters.

One important property of communication is *statically ordered*. This means that the relative order of messages on each tile is specified at compile time and holds by induction because the compiler fixes the messages it receives.

Dynamic Network

The dynamic network uses a dynamic switch instead of worm-hole routing by making decisions based on the dynamic network is used when either the source or destination is a dynamic network at compile-time.

The dynamic network is much slower than the static network for several reasons. First, messages on the dynamic network require additional information, such as the destination address, which must be constructed, inserted into the network header, and then the destination tile, using processing resources in every switch. Second, the order of messages on the dynamic network is not fixed, so it is required to use expensive mechanisms, such as a buffer, to store messages. Third, due to the lack of flow control on the dynamic network, messages must be spilled to memory if more arrive at a tile than the queue can hold.

Since RAWCC chooses the assignment of instructions to tiles, the scheduling between instructions can be performed on the static network.

2.1.3 Distributed Memory

The cache memory for Raw processors is distributed, which improves the scalability of the Raw architecture in many ways. First, by splitting into multiple banks, the wires are kept short and latency is low. Second, since the memory banks are distributed, the compiler or software may be able to take advantage of local data and cache references, and the banks they refer to close to the processor, reducing latency. Third, each tile that is added to the architecture increases capacity and bandwidth without sacrificing cache locality. Instructions can be parallelized while preserving locality. Fourth, power and memory bandwidth can be used more efficiently, reducing latency due to long wires and complex arbitration. Finally, compared with monolithic memory structures,

Raw is a *bank-exposed architecture* [5], meaning that the memory banks are exposed to the software, and that memory references are resolved to specific memory banks at compile-time. Bank-exposed architectures have several advantages over hardware-based unified systems: they avoid the overhead associated with hardware arbitration logic and the complexity of the advantages are only gained when most of the memory accesses are *bank-biased*, meaning that they reference known banks. In a bank-biased architecture, the arbitration logic, the dynamic network in the tile, and the dedicated hardware arbitration logic, the compiler can optimize each memory instruction refers to is critical. RAWCC uses loop unrolling and alignment as a technique to achieve *bank disambiguation*.

that maximum memory bandwidth can be achieved. RAWCC must satisfy these criteria: *equivalence class unification*.

2.2.1 Equivalence Class Unification

Equivalence class unification [5] begins with a pointer analysis. SPAN [15]. SPAN assigns a unique *location set number* to each memory access in a program. An abstract object is either a variable or a group of dynamic objects created at a single memory location. Elements of a single array belong to the same abstract object. Elements of a struct are each considered separate objects. To map memory references to abstract objects, SPAN assigns to each memory reference a *list* containing the location set numbers corresponding to the memory reference. The memory reference instruction may refer to

After the pointer analysis is completed, RAWCC identifies *equivalence classes*. Alias equivalence classes are defined by location set numbers such that each memory reference belongs to only one equivalence class. To find these classes, RAWCC generates a bipartite graph with one node for each memory reference. Edges are added to the graph between memory references whose abstract objects whose location set numbers are in the same equivalence class. RAWCC then finds the connected components of the graph. Each component constructs one equivalence class component.

Finally, RAWCC assigns each equivalence class a unique *equivalence class number*. Memory references in different equivalence classes may never refer to the same memory location.

is forced to map each array to a single tile, resulting in loops. RAWCC uses loop unrolling and array disambiguation and memory bank load balancing expressions that are affine functions of inner loop indices.

When such an array is discovered, RAWCC maps it to memory banks in a round-robin fashion. The minimum number of iterations required to guarantee the unrolled loop can be bank disambiguated.

On a software-exposed architecture, such as the one used for ILP exploitation in two ways. First, the loop unrolling across memory banks means that multiple elements are accessed. Second, the unrolled loop produces a larger number of instructions which gives the compiler a greater opportunity for ILP. However, to achieve maximal performance benefit, the time scheduler must generate code that effectively exploits the increased opportunity for locality and disambiguation of memory bank disambiguation.

2.3 Space-Time Scheduler

The space-time instruction scheduler in RAWCC schedules instructions in each forward control flow region by identifying sources and ordering them in time so that the region completes as quickly as possible. Since this scheduler exploits and exploitation of ILP, its effectiveness is critical.

lel streams, while placement is the related task for
 individual tiles. To partition the instructions into
 instructions that have no exploitable parallelism for the
 target processor. Then clusters with high level of parallelism are
 merged until the number of clusters equals the number of processors.
 placement, RAWCC then creates a bijection between the clusters and
 instructions using a greedy algorithm that attempts to minimize
 communication over the entire schedule.

After the instructions have been spatially partitioned, the
 scheduling algorithm to construct the temporal schedule. RAWCC
 is also responsible for scheduling the communication between
 instruction precedence and data dependence. The final output is
 to produce a computationally correct, deadlock-free schedule.

This approach to spatial instruction scheduling has one significant
 it has one significant shortcoming. Many of the instructions are
 disambiguated memory references that must be resolved before
 this system, RAWCC is unable to efficiently schedule these
 instructions when generating a schedule. Due to the nature of the
 communication network, long communication delays are incurred
 results of these memory references are transmitted back to the
 processor.

This system also suffers from the phase of the problem. The prob-
 lem is a result of the division between spatial and temporal
 partitioner seeks to minimize communication. The spatial
 ates partitions that cannot be executed in parallel. The temporal
 instead. This defeats the purpose of partitioning.

poral scheduling information into account whe

Empirical Analysis

Characteristics

The problem of performing optimal space-time exposed architectures is NP-hard. Since a solution is unlikely, a heuristic approach leading to this chapter the structure of the individual space-time applications is examined and analyzed. This chapter presents program characteristics critical to effective space-time needs presented by these characteristics are the space-time phase ordering problem.

3.1 Benchmark Suite

Table 3.1 describes the set of benchmark applications. With the exception of SHA, they were extracted from the SPECint abench [11] benchmark suites. The version of the benchmarks by Matt Frank. The benchmark set includes several applications, and an application without regularity.

With the exception of Life-static and Mxm, no changes were made to the benchmarks, though a small number of changes

	Matrix	(Spec92)		
Tomcatv	Dense Matrix	Spec92	FORTTRAN	2
Vpenta	Dense Matrix	NASA7 (Spec92)	FORTTRAN	1
Mxm	Dense Matrix	NASA7 (Spec92)	FORTTRAN	
Adpcm	Multi-media	Media-bench	C	3
SHA	Multi-media	Matt Frank	C	6
Fpppp-kernel	Irregular	Spec92	FORTTRAN	10
Jacobi-small	Dense Matrix	Rawbench	C	

formed. Currently the Raw architecture does not support floating point arithmetic, so all floating point operations are emulated in software. In addition, data set sizes were reduced to fit in the cache of the target architecture. The data set sizes used for each benchmark in the Raw benchmark suite are shown in Table 3.1.

Since the outer loop of Mxm had already been unrolled for good performance on VLIW machines, this unrolling was not done for the Raw machines. The performance on 8 and 16 tile Raw machines was not as good as the performance on the VLIW machines. This was due to the fact that the code contained a disproportionately large number of dynamic references. These references were made static through the use of the `static` keyword.

3.2 Visualizing Scheduling

A direct approach to understanding the structure of the code is to look at them. Through visual analysis, the characteristics of the problem can be examined in detail. Heuristic scheduling characteristics can then be developed to approximate the optimal schedule.

For visualization purposes it is convenient to have a visual representation of the code.

a directed acyclic graph with nodes representing instructions and edges representing dependencies. To simplify the layout, only true data dependencies that produce values to instructions that use those values are shown. Data dependencies are omitted from the visual representation. This simplification often obscures the underlying structure of the graph.

Once the scheduling region is in the proper form, the layout engine system is used to generate an image. When color printing, nodes can be shaded with different colors and edges with different line styles. In this chapter, however, nodes are distinguished by their shape. Figure 3-1 enumerates the shapes of the particular tiles in this thesis.

Many of the scheduling regions studied in this thesis consist of instructions that copy a value for the use of a later instruction. Figure 3-2 shows a region with this behavior for eight tiles. Removal of the edges representing data dependencies reveals the graph structure and allows the layout engine to produce a cleaner image as seen in Figure 3-3. Whenever a graph for a scheduling region is shown in this way, the caption will contain a string of the form "Graph of copy instructions whose outgoing edges have been removed." All graphs in this thesis contain directed edges. The layout of the full-page representations of all scheduling regions is determined by the aspect ratio of the tiles.

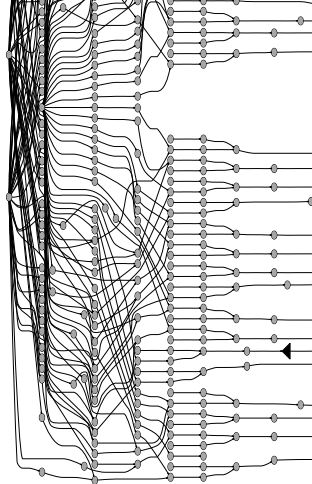


Figure 3-2: Chol

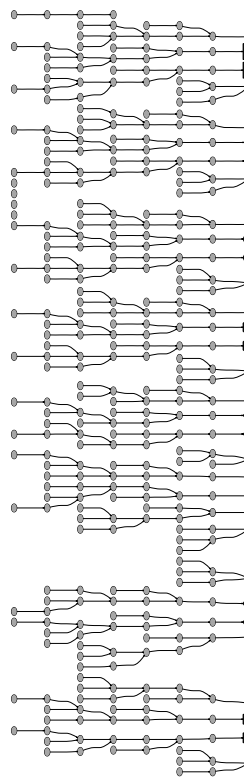


Figure 3-3: Choles

tend to spend the majority of their execution optimizing the schedules for the bodies of the execution. Second, the innermost loops are the scheduling regions derived from these inner loops for each application and provide the greatest benefit to each application.

3.2.2 Dense Matrix Application

All of the dense matrix applications examined exhibit a scheduling region similar to the region taken from Mxm, compiled with the same type of scheduling region is the result of performing a loop that indexes an array or set of arrays using affine expressions. The eight unrolled loop iterations present in the region are far from each other after the initial copy edges are added. There are no dependencies between unrolled iterations of the loop, as long as the memory usage pattern does not change.

The scheduling region shown in Figure 3.10, taken from the Mxm tiles, exhibits similar structure to the previous scheduling region. Mxm. Once again, the loop has been unrolled to eight iterations. Two multicast nodes with nearly 20 children are added to the graph to simplify the layout.¹ This region has been taken from the original loop, all static memory references are preserved.

¹The communication code generator for RAWCC was modified to be sent to each tile, and since the children of these nodes are tiles, the communications become simple broadcasts.

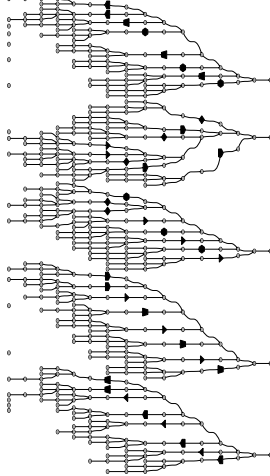


Figure 3-4: Life

possible to execute this entire region without a static network after the initial multicasts have been done by the Cholesky, Tomcatv, and Vpenta benchmarks.

3.2.3 Multimedia Applications

Unlike the parallel structure exhibited by the multimedia applications examined in this thesis, extensive scheduling regions taken from Adpcm, Figure 3-8 respectively. Any exploitable ILP is sparse and less regular structure than the available parallelism in applications. These applications will therefore do not benefit from increases in the number of available processors. In the scheduling region in SHA, we estimate the amount of parallelism.

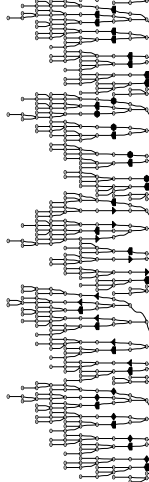


Figure 3-5: Mxm

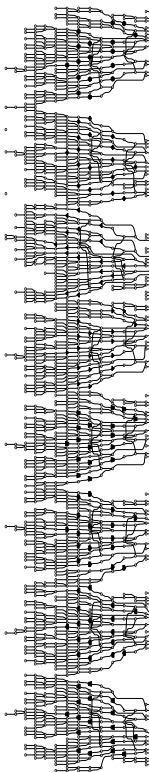


Figure 3-6: Vpent



Figure 3-7: Adp

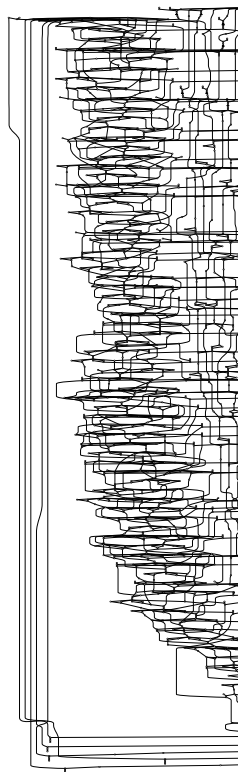


Figure 3-8: SH

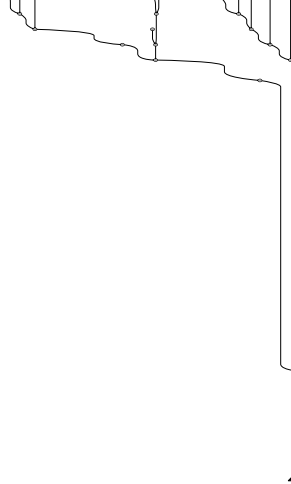


Figure 3-9: Fppp

3.2.4 Irregular Applications

The Fppppp-kernel is the only application exhibiting high memory usage. Figure 3-9 displays the single iteration of the Fppppp-kernel. While the kernel contains a large amount of memory usage, the ILP found in dense matrix applications. The Fppppp-kernel is the long reduction that dominates the lower bound. This reduction is converted from its linear form to a tree to reduce the memory usage of this region when it is compiled for a large

3.3 Discussion

Visual examination of scheduling region graphs is a useful tool for solving the space-time scheduling problem for software

to the tile containing the memory bank it references, *pre-placed instructions*. If instruction pre-placement is performed without performing spatial scheduling, a large amount of delay is the result.

The instruction partitioning module in the current implementation ignores instruction pre-placement information. The current module considers the locations of pre-placed instructions in relation to tiles, the pre-placed instructions are not necessarily neighbors due to uninformed decisions made by the compiler. The delay associated with memory references, made by the tiles, is difficult and potentially eliminating some of the benefits of parallelization. Also, since these memory references are not contiguous of the original loop iterations, network congestion is likely. If the tiles attempt to perform the remote memory references, it suggests that dividing the instruction assignment into two phases is inappropriate in the presence of pre-placed instructions.

Conveniently, the pre-placed instructions can be used in the assignment analysis often provide useful information about the memory region. For unrolled loops without loop-carried dependencies, memory references are spread across the breadth of the loop. Starting locations for the search for parallelism can be determined. If a loop iteration contains disambiguated references, the pre-placed instructions immediately suggest a starting point for the search. In application like Mxm, the pre-placed instructions are not ideal, though they still make a good starting point.

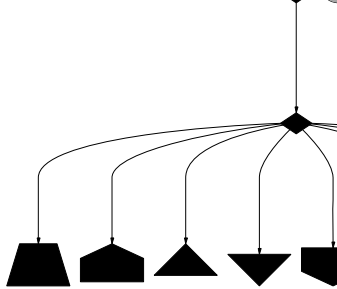


Figure 3-10: A scheduling region with a

The serial nature of the multimedia application's memory-access structure, means that the performance in the partitioning of their scheduling regions and the placement of instructions throughout the regions does not matter as much as in other tasks, however.

As the only disambiguated memory reference occurs at the very end of the scheduling region, pre-placed instructions are not needed for space-time instruction scheduling of this application.

3.3.2 Critical Path

A *critical path* in a scheduling region is a linear sequence of instructions whose total latency is the maximum over all such sequences. Figure 3-11 shows a small scheduling region from the Chameleon benchmark with a critical path displayed as diamonds. On a system with unlimited computation resources and zero communication latency, the execution time for a scheduling region will be exactly the length of the critical path. By definition, instructions on the path cannot be

inter-tile communication on the critical path of the path. This suggests that an instruction, such as SHA, may achieve good results on a single tile, and then searches for groups of instructions that can be assigned to a different tile without increasing the critical path.

The critical path plays a different role in tiling for parallelism, which includes all of the dense matrix operations. In these applications, each separate loop iteration is an important consideration is that the loop iteration order is not important. While the temporal scheduler must observe the order of instructions assigned to each tile execute as efficiently as possible, it instead focus on the high-level structure of the computation. The critical path to partition and then concentrate on the critical path, it may miss the opportunity to take advantage of parallelism at the loop iteration level.

The Fppppp kernel demands a hybrid approach. If a large part of the scheduling region contains a large critical path, a path-based system may not make efficient use of the hardware. If it is dominated by a reduction which presents a large critical path, it is concerned with discovering ILP, it may distribute the critical path among the tiles and introduce many unnecessary instructions. A portion of the schedule that is dominated by a single instruction, a partitioner for this region must be aware of the critical path. A partitioner around the critical path, and how to partition the schedule appropriately.

eration given to the locations of pre-placed instructions. Each instruction is assigned to a single tile, preferably the tile closest to the beginning of the reduction.

3.3.3 Register Pressure

The Fppppp scheduling region contains 28 nodes, each with 228 values. Most of these nodes have few, if any, children, and are scheduled early in the computation. Since there is a large amount of ILP, each of these nodes is likely to be scheduled on many tiles. This has two implications. First, it is likely that the start of the schedule as these instructions come out of the compiler. Second, each value is replicated by the number of tiles it is scheduled on. The schedule also suffers from high register pressure. A scheduler that is too aggressive in seeking ILP may cause a performance loss, due to the spilling of these values to memory. A less aggressive approach.

This mistake can also be made by the tiling process. Compilers often interleave instructions from unrelated regions, leading to high register usage in the presence of long-latency instructions. A scheduler that is too aggressively when it is presented with a region of high ILP, such as those found in Fppppp or Vpenta, the scheduler may schedule more intermediate values than it can store in registers, leading to spilling in memory and drastically slows the execution.

their tradeoffs is extremely daunting. The challenge is to divide the space-time scheduling and register allocation and give each phase responsibility for ensuring that tradeoffs are taken into account. Unfortunately, this is a hard problem.

The probabilistic scheduling framework proposed in this paper aims to minimize the effects of the phase ordering problem by combining simple space-time scheduling and register allocation with a system that is both flexible and effective. This is achieved by two algorithms, or phases, two new abilities: they can take into account the previous work of other phases and make decisions that can express their level of uncertainty with a confidence value.

4.2 Detailed Description

The combined-phase probabilistic scheduling framework consists of three main components: a probabilistic schedule representation, the current schedule state, a set of small heuristic allocation algorithms that modify the schedule, and a set of algorithms to apply to the schedule in order to update the state.

4.2.1 Probabilistic Schedule

A conventional schedule representation maps each instruction to a resource, time slot, and register. The probabilistic schedule representation by associating with each instruction a probability of being scheduled in a particular resource, time slot, and register.

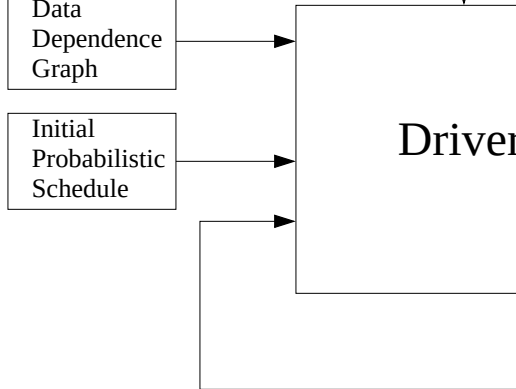


Figure 4-1: Combined-phase probabilistic scheduler.

and registers. This distribution describes the probability of assigning an instruction to each (processing resource, time slot, register). This approach is discussed in terms of probabilities, decision-making, and feedback. This approach are rarely random, especially when using a probabilistic schedule. If this step was purely probabilistic, the schedule would worsen due to low-probability instructions to be scheduled at inappropriate times or conflicting registers based on the assignments of previous phases.

This generalized representation gives the scheduler more flexibility than a conventional representation. By incorporating the decisions made by previous phases, along with feedback to the schedule. Also, if one phase determines a particular idea, it can reduce the probability of that assignment, thereby increasing the probabilities of any alternative assignments.

In addition to modifying the work of previous phases, the scheduler are also given more control over how they express their decisions.

4.2.2 Local Heuristics

Local heuristics are small simple algorithms that improve the probabilistic schedule. Each heuristic adds a specific piece of knowledge to the schedule. By combining heuristics, knowledge about many different aspects of the register allocation problem is built up to produce a better schedule. A heuristic, does not mean that the algorithms are local to a scheduling region. Instead it means that each heuristic has a specific area of knowledge that is local to their area of the problem.

The heuristic algorithms used for combining phases generalize conventional compiler back-end phases. First, they create a probability distribution over space and time for each instruction. Second, they may examine previous phases directly related to their task. This differs from traditional phases perform non-overlapping tasks, such as register allocation.

There are three general types of algorithms used in this approach. Improvers apply local scheduling heuristics to an efficient schedule. Selectors make specific decisions in order to make the schedule more concrete. Constrainters add constraints to the schedule in order to insure

For example, a critical path improver may find a critical path in the DAG graph and give all of the instructions on it higher priority than the others in those instructions. A communication improver may change a communication probability map so that it is more likely to be assigned to a processor than its neighbors. A register allocation improver may find many live ranges and attempt to rearrange instructions to reduce register pressure.

Selectors

Selectors ensure that the probabilistic scheduler produces a valid schedule. Without the intervention of selectors, the schedule may stay wide and flat indefinitely, with no progress being made. A selector alleviates this problem by iterating over the entries in their probability maps based on selection criteria. For example, a single-tile selector may increase the probability of the highest time, tile, and time slot. A maximum-tile selector may choose to increase the probability of the tile with maximum total probability for a given time slot. A time-slot selector may increase the probability of an instruction in its tiles in its median time slot.

Constraint Enforcers

Constraint enforcers are responsible for preventing the scheduler from outputting an invalid schedule or spending too much time.

of the schedule and ensure that the other heuristics are not spending too much time improving impossible schedules.

4.2.3 Application of Local Heuristics

Because this scheduling approach employs a search over space-time schedules and register allocations, it is a combinatorial problem. If each local heuristic is applied to a schedule in a fixed order, the phase ordering problem will be solved the same way as it does with conventional scheduling approaches. In the phase ordering problem the local heuristics must be applied a large number of iterations is large and the order of the heuristics of going first or last or immediately before or after the other is the decision of which heuristic to apply at which point in the search. The effectiveness of this approach.

The primary responsibility for choosing the next heuristic to apply to the schedule is given to the *driver*. Through the application of which local knowledge should be applied to the current schedule. If the driver performs well, the solution will improve with each iteration. In addition to the current schedule, a probabilistic schedule at the start of the process is maintained when it determines that the schedule is complete.

The principle task of the driver is to choose the next heuristic to apply to the schedule in order to ultimately produce an efficient schedule. Iterating over a fixed set of algorithms for a fixed number of iterations. Sophisticated approaches are likely to produce a better schedule.

that were able to improve the schedule in pre

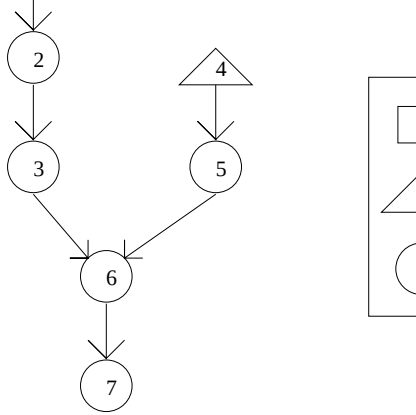
The decision by the driver to terminate
be based on a fixed list of algorithms to run
possibility is to measure the quality of the sc
stagnates or worsens for a period of time.

When outputting the concrete schedule, th
on the probability distributions in the proba
free to use any selection criteria to make t
time, tile, register triple with maximum proba
is completely general. If different selection cr
median time slot, a selector that uses the prop
preferred entry has maximum probability im
is issued.

4.3 Example

Figure 4-2 gives a simple example of the oper
scheduler. To simplify the example, only sp
multiple heuristics interact to produce the
contains two tiles and communication of a wor
The lower portion of Figure 4-2 displays the
heuristics are applied to the schedule. Large c
instruction to be assigned to that tile while s

When the schedule is initialized, each ins
the tiles. Then the critical path heuristic id



Probability distribution

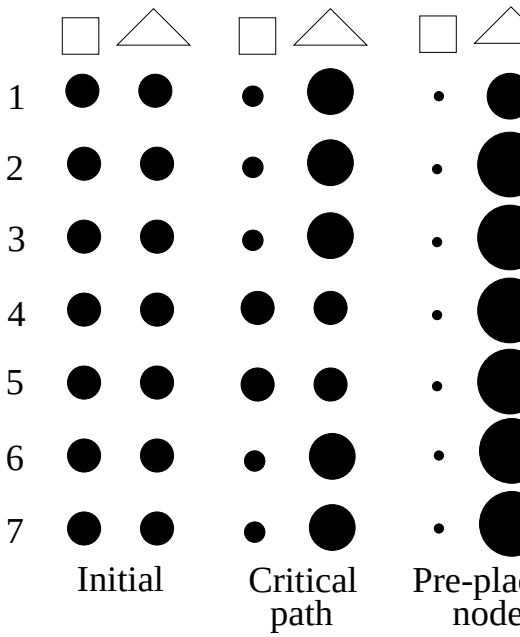


Figure 4-2: Simple

is executed again, except that this time tile 1
this point, the driver decides that the schedul

scheduler. For each instruction, the space-time distribution is a uniform distribution over all tiles and all time slots. The principle is that any instruction can be scheduled. The set of possible tiles and time slots is derived from the instruction to root and leaf instructions.

Each dimension of the probability map is a vector in the space for commonly accessed sums of probabilities. The perimeter of the matrix, sums over each node, tile, and time slot, and tile and time slot can be maintained. The probability map with each node, tile, and time slot triple. This is updated over any combination of node, tile, and time slot. The updates run in constant time, the old value to the new value and this difference is added to each of the entries of the entry being updated.

One important feature of the representation is normalization. For each instruction, the sum of the probabilities in the distribution should equal 1. For performance, the probability map for an instruction every time the instruction maps are only normalized when the driver or compiler performs the operation.

The probabilistic schedule representation simplifies common tasks. These include finding the tiles and time slots for a given instruction, finding the median time slot for an instruction, and finding the tiles visited by an instruction at a given time slot. Functions are also provided to calculate the constituent values, by a given factor.

such that

$$p_{nts} = Pr[\text{instruction } n \text{ is sched}$$

Next define

$$p_{nTs} = \sum_{t=0}^{T-1}$$

$$p_{NTs} = \sum_{n=0}^{N-1} \sum_{t=0}^{T-1}$$

$$p_{NTS} = \sum_{n=0}^{N-1} \sum_{t=0}^{T-1}$$

and so on. Note that $p_{nTS} = 1 \forall n$.

5.2 Heuristics Implemented

This section contains a list, and short descriptions, of the heuristics implemented for the probabilistic scheduling approach.

5.2.1 do_preplaced_nodes

This improved heuristic gives instructions an initial preference to be assigned to the same tile as their nearest pre-assigned instruction. First the heuristic iterates over the tiles and calculates the distance in the dependence graph from each instruction to the nearest pre-assigned to that tile. Then it iterates over the instructions pre-assigned to a particular tile the heuristic assigns a preference to being assigned to any other tile to zero. If the

```

    If n is preplaced on tile s then
        Set  $P[n][T][s] = 1$  and  $P[n][T][s]$ 
    Else
        For each tile s do:
            Set  $P[n][T][s] = 1 / \text{min. dist}$ 
        End
    End If
End

```

5.2.2 balance_tile_load

This improver heuristic iterates through the total weight assigned to each tile in each time slot multiplicatively scaling the probabilities for lightly

```

balance_tile_load()
    For each time slot t do:
        For each tile s do:
            Set  $P[N][t][s] = P[N][t][S] / S$ 
        End
    End

```

5.2.3 round_robin_tiles

This selector assigns instructions to tiles in a round robin fashion with a fixed probability of assignment to the selected tile

End

5.2.4 strengthen_max_tiles

This selector heuristic iterates through the instructions in the instruction set, increasing the instruction's probability on the tile it already has highest probability on.

`strengthen_max_tiles()`

For each instruction i do:

Let s = maximum probability tile for i

Set $P[i][T][s] = 1.5 * P[i][T][s]$

End

5.2.5 find_parallelism

This improver heuristic iterates over the time slots, finding the time slot with the least parallelism usage in time slots with high load imbalance. It then finds the tile with the highest total weight in that time slot and reassigns it to be cases where the tile with highest total weight is less than half the weight of the least heavily loaded tile. The tile may be allocated to the heavily loaded tile if the pair of nodes that are the greatest distance apart in the computed distances. If these nodes are sufficiently close to each other, the tile is reassigned to its near neighbors, has its probability for that time slot increased, and the tile is reassigned to the most lightly loaded tile in that time slot.

`find_parallelism()`

For each time slot t do:

```

        If distance between i and j > threshold
            Let p = P[i][t][l]
            Set P[i][t][l] = P[i][t][b]
            Set P[i][t][b] = p
            For each neighbor n of i do:
                Let p = P[n][t][l]
                Set P[n][t][l] = P[n][t][b]
                Set P[n][t][b] = p
            End
        End If
    End If
End

```

5.2.6 examine_neighbors

This improver heuristic iterates over the instructions in the instruction set and improves communication by placing them on the same tile. For each instruction, it counts the number of neighbors of an instruction that are on the same tile. Then, for each tile the heuristic multiplies the probability of placing the instruction on that tile by the number of neighbors that were counted for that instruction.

```

examine_neighbors()
    For each instruction i do:
        For each tile s do:
            Let C[s] = 0
        End
    End

```

```

        Set  $P[i][T][s] = P[i][T][s] * C$ 
    End
    Let Total =  $P[i][T][S]$ 
    For each tile s do:
        Set  $P[i][T][s] = P[i][T][s] / \text{Total}$ 
    End
End

```

5.2.7 highlight_critical_path

This selector heuristic begins by building a set of critical paths in the data dependence graph. If any instructions are pre-placed, the critical path is assigned to the pre-placed instructions. Otherwise the critical path is assigned to the

```

highlight_critical_path()
    Let C = a critical path in the dependence graph
    Let s = least weighted tile
    For each instruction i in C (in topological order)
        If i is pre-placed Then
            Let s = tile for i
            Break
        End If
    End
    For each instruction i in C (in topological order)
        If i is pre-placed Then

```

5.3 Driver Implementation

The driver used for this implementation is scheduling heuristics to the probabilistic scheduling instruction to the tile with maximum total profit.

This primary design goal for this driver was to iterate over the local heuristics many times over. If heuristics were properly tuned for multiple iterations over a set of scheduling regions, the driver needs to improve the scheduling process. At the minimum, the driver needs to know on the amount of recent improvement made by each heuristic to choose which heuristic to run based on the relative efficiency and effectiveness of the scheduling process.

Chapter 6

Implementation and

This chapter presents an implementation of the probabilistic schedule described in Chapter 5 targeted for the Raw architecture. The performance results were obtained using the Raw architecture of the Raw architecture described in section 2.1.

6.1 Implementation

The probabilistic schedule in this implementation is for an array of double precision floating point values, a tile number, and tile number.

Of all combinations and orders of scheduling, the most efficient schedules were generated by the `balance_tile_load`, and `strengthen_max_tiles`. The results of this thesis were obtained using this driver. Part of the results of all of the benchmarks had pre-placed nodes to guide the scheduling hints. Any additional work after the initial schedules rather than improve them.

placement, spatial instruction scheduling, to graph-coloring register allocation.

Application performance results were obtained using the simulator. This simulator properly models the compiler, as well as the execution activities of the hardware. Results for the suite of benchmarks detailed in Table 6.1 match exactly with an extended MIPS R4000 core and the results for arithmetic instructions are listed in Table 6.2.

Speedup values are derived from comparison of the execution time of a benchmark compiled by the Machsuif [18] MIPS R4000 compiler on a MIPS R4000 core located on a single Raw tile.

Section 3.1 describes the set of benchmarks used in this chapter presented in this chapter.

6.3 Speedup

Figure 6-1 presents the speedup from compilation and execution of probabilistic partitioning and RAWCC with probabilistic partitioning. Cycle counts and speedup from probabilistic partitioning are given in Table 6.1. Probabilistic partitioning on most of the benchmarks.

Both versions of RAWCC demonstrate speedup for all benchmark applications. Probabilistic partitioning is faster than conventional partitioning on all dense matrix benchmarks except for Vpenta on 16 tiles, where probabilistic

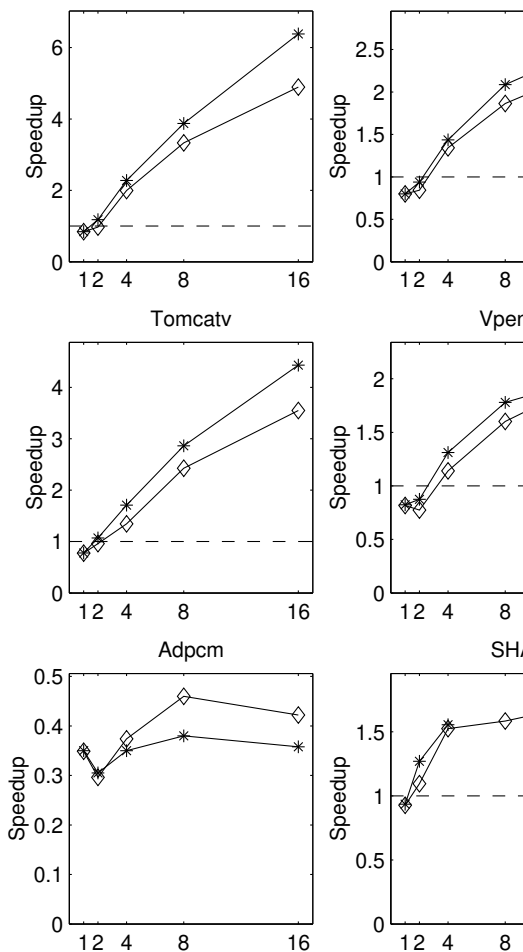


Figure 6-1: Speedup of benchmarks on Raw v benchmarks compiled with Machsuif for MIP

Number of tiles		01	
Life-perf	Conv. part.	346,568	
	Prob. part.	347,011	
	speedup	0.999	
Cholesky	Conv. part.	136,735	
	Prob. part.	136,737	
	speedup	1.00	
Tomcatv	Conv. part.	337,279	
	Prob. part.	334,694	
	speedup	1.01	
Vpenta	Conv. part.	214,168	
	Prob. part.	212,602	
	speedup	1.01	
Mxm	Conv. part.	1,893,783	1,
	Prob. part.	1,893,783	1,
	speedup	1.00	
Adpcm	Conv. part.	973,051	1,
	Prob. part.	972,028	1,
	speedup	1.00	
SHA	Conv. part.	1,300,544	1,
	Prob. part.	1,290,816	
	speedup	1.01	
Fpppp-kernel	Conv. part.	1523	
	Prob. part.	1552	
	speedup	0.981	
Jacobi-small	Conv. part.	47,052	
	Prob. part.	47,052	
	speedup	1.00	

the quadratic space requirements of the chosen kernel. Both partitioning algorithms perform poorly on kernels with many dynamic memory references.

The results on Fppppp-kernel illuminate the performance of the partitioning algorithm on pre-placed instruction region structure. Fppppp-kernel contains explicit pointers to nodes to serve as guides, the probabilistic partitioning

Analysis

This section presents an analysis of the probabilistic spatial scheduler implementation described in Section 6.1.

7.1 Potential Problems

This combined-phase probabilistic scheduling approach presents several challenges that must be addressed in a successful implementation. These challenges include working with unconditional probabilities, ensuring convergence in a meaningful way, finding the right balance between overly aggressive and overly passive heuristics, and composing the scheduler to be positively affected by the running of multiple parallel schedulers.

7.1.1 Unconditional Probabilities

One of the most challenging aspects of the probabilistic scheduler is the lack of conditioning between the probability distributions of different operations on groups of instructions.

Consider the critical path heuristic. The probability of selecting a tile is to place all instructions along the critical path of that tile is unimportant. What the critical path is depends on the state of the system, which is not captured by the heuristic.

The critical path heuristic is left with the task of how to express that the critical path can be placed on any of the critical path uniform distributions across the tiles. This fails to form any sort of link between instructions and the intended purpose of the critical path heuristic. The critical path conditional probabilities unconditional by choice of tile. The critical path instructions to that tile. While the critical path heuristic of “deconditioning” conditional probabilities is a solution to the irreversible decision problem that the probabilistic scheduling system has to overcome. If the probability distributions were conditioned on a single instruction in the tile, the assignment of that instruction would change the assignment of the tile. The irreversible decision problem would be alleviated. The probabilistic scheduling system cannot represent the critical path instructions explicitly and care must be taken to ensure that the critical path instructions are always placed consistently with those in the tile.

One possible solution is to represent the critical path instructions as heuristics. For example, the critical path heuristic could be used to ensure that the critical path instructions to act as indicators, and ensure that the critical path instructions are always placed consistently with those in the tile. The driver must cycle through the heuristic set and the conditional relationships are represented.

7.1.2 Schedule Convergence

Another challenge that must be addressed when using a probabilistic scheduling system is the need for the schedule to converge to a stable state.

heuristics, dominate the scheduling process and make poor decisions in order to make progress. While selectors can stimulate the scheduling process, developing a good set of selectors seems desirable.

7.1.3 Heuristic Cooperation

Tuning the heuristics so they work well with each other is a consideration. When heuristics are either too greedy or too conservative they produce suffer. Heuristics that are overconfident in the decisions of other heuristics without regard for their own may have been more appropriate for the current situation. In this manner are not cooperating, and this can lead to poor behavior.

On the other hand, if the heuristics are too conservative, small changes to the schedule probabilities, likely to be made by selectors, and while selectors are ill-suited for the task of producing good schedules.

Machine learning techniques may be able to help with early tuning heuristics for efficient cooperation. This is a problem across the space of tunable parameters. Machine learning can discover patterns in the space to aid solution finding. It can explore more of the space than would be possible with manual tuning.

by the bad decisions of others, the resulting solution means that at some level there must be communication; they even need to be aware of each others' goals.

Local heuristic algorithms developed for the front end have more structure than conventional back end compilation. One requirement is that they must be able to examine knowledge from other algorithms and effectively integrate this knowledge. This area is key to good heuristic composition.

A second area in which heuristic algorithms are needed in compiler phases is that they must judge their own results in terms of a confidence measure. Furthermore, they may express some confidence in alternative solutions. Heuristics already employ cost functions in this area. Good ideas may come easily. Suggesting good alternatives

Related Work

8.1 MIMD and VLIW Compilers

The space-time scheduling problem for Raw is also faced by compilers for Multiple Instruction Multiple Data (MIMD) and Very Long Instruction Word (VLIW) architectures. Many of the compiler algorithms are borrowed from work targeted for the Raw architecture.

The division between spatial and temporal scheduling is partially motivated by Sarkar's work with MIMD machines. He also separates spatial and temporal scheduling in the assignment problem because their targeted machines are MIMD. The MIMD scheduling problem has been studied in the literature. Some of some representative work. One major difference between Raw and MIMD machines is the presence of a global memory in the Raw scheduling problem. The predetermined assignment problem is the primary focus of this project, does not address the scheduling problem.

The Bulldog [8] compiler faces a scheduling problem. It targets a VLIW machine with multiple processors, files, and memory banks. Thus Bulldog must assign processor mappings for memory accesses. Bulldog is divided into two phases, assignment and scheduling. The assignment phase is the primary focus of this project, does not address the scheduling problem.

of the dependence graph.

Chapter 9

Conclusions

9.1 Summary

In this thesis the structure of scheduling regions was studied empirically. This study revealed the structure of scheduling regions; however, most by a small number of specific features. These are exploited by simple scheduling heuristics, the for a single heuristic or composition of heuristics.

Following the analysis of scheduling regions facilitates the use of local heuristics to iterative structure is based on a probabilistic schedule representation share a rich knowledge base and an iterative heuristic to make multiple contributions to the local knowledge is collected and combined to

A simple implementation of a spatial scheduler this architecture. While much of the implementation system already shows promise on most benchmarks pre-placed instructions are critical constraints

First and foremost, the probabilistic scheduling addresses one phase of the code-generation task. An extension to this work is to implement temporal scheduling in the probabilistic scheduler as well. One possible extension is to generate priorities for the instructions and pass those priorities to the temporal scheduler. Another solution is to implement the list scheduler as well and then develop a set of improvers that work on the output of the list scheduler.

A more ambitious extension to this research is to develop a probability distribution over the register set for each instruction that is independent of the space-time probability distribution. Additional heuristics that perform register allocation are added to complete the phase-unification aspect of the scheduling system.

9.2.2 Better Heuristics and Tuning

One area in which this research should certainly benefit is in finding better heuristics. While the current set of heuristics works on the benchmarks there is much room for improvement.

Each heuristic is intended to add some piece of information to the structure of the solution. Heuristics that work on applications lacking pre-placed nodes, such as the current set, are less effective. Also, heuristics that are not specific to the effectiveness of this system. Also, heuristics that are not specific to the target architecture, such as cache misses, alternative instruction sets, and so on, are less effective on the target architecture.

Finally, the driver is another source of interest. Currently the implemented driver does not exist for the sake of outputting it. The driver could choose heuristics based on their previous performance of the scheduling region. If the schedule is sparse, a load-balancing heuristic. If the schedule is overfull, one that swaps blocks of instructions between tiles to streamline flow. If the schedule contains too many registers, that reduces register pressure. This type of adaptation works well over a much wider range of benchmarks than the extension of the current driver in this area is limited, if they exist and are distributed throughout the system. heuristic. If pre-placement information is not available, helpful information, the driver may use a different heuristic. makes some seed decisions at the beginning.

Another interesting way in which the driver could use a type of machine learning algorithm to discover heuristics to produce good schedules. Genetic programming has produced results when applied to this system.

Graphs

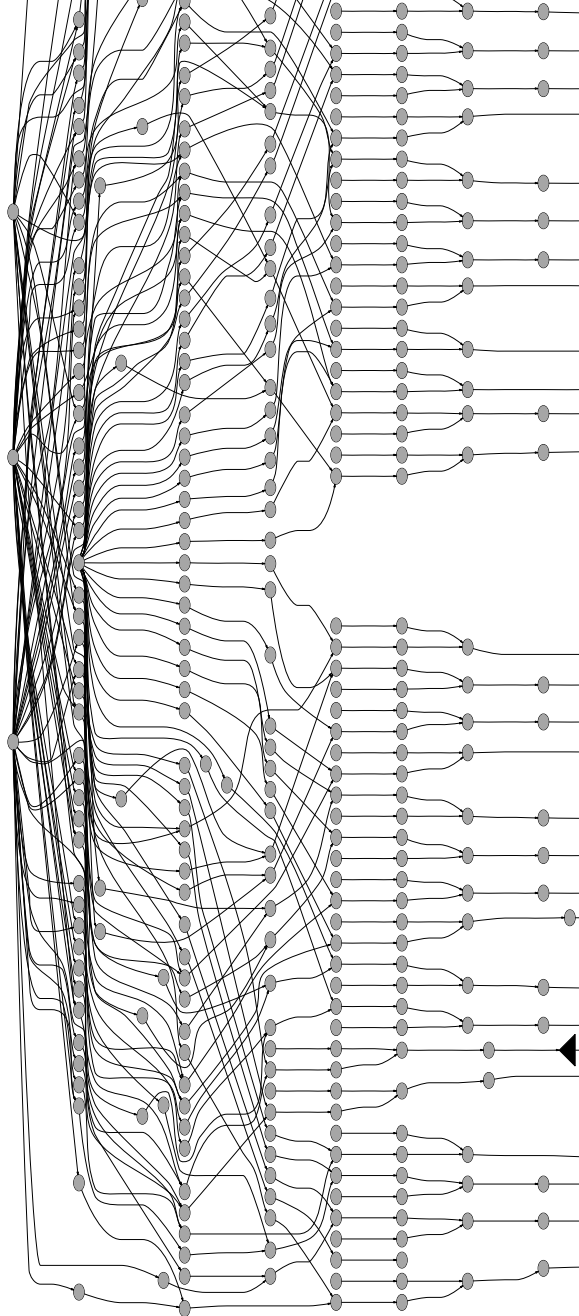


Figure A-1: Chol

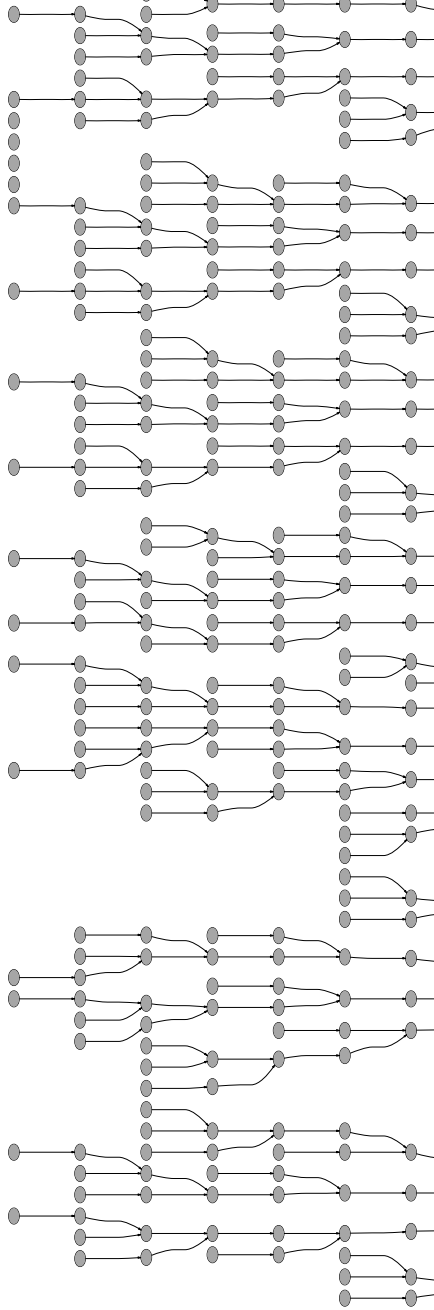


Figure A-2: Choles

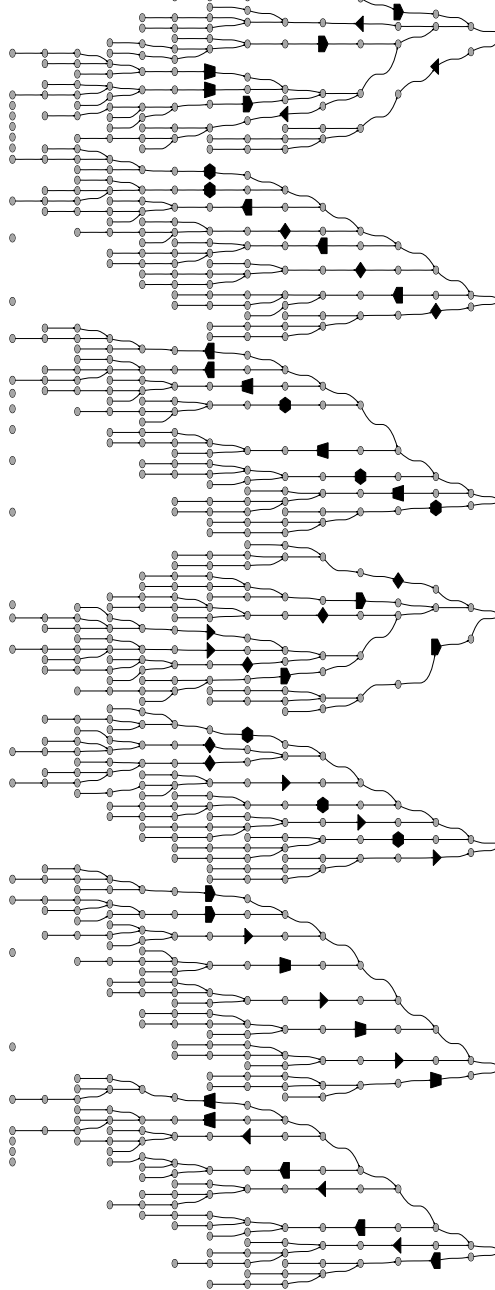


Figure A-3: Life

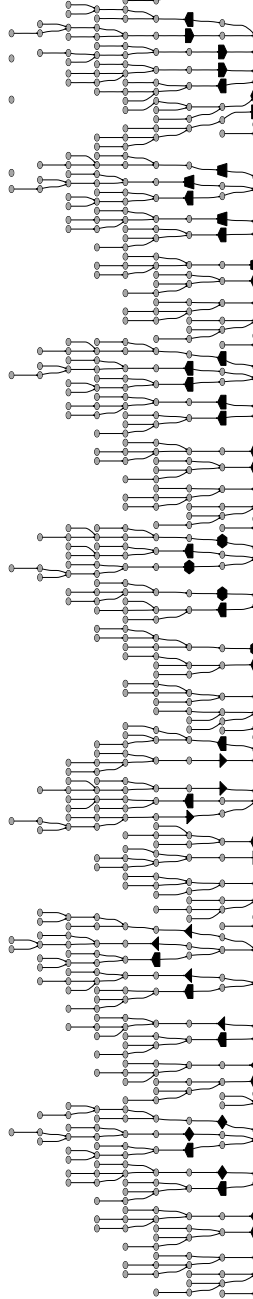


Figure A-4: Mxm

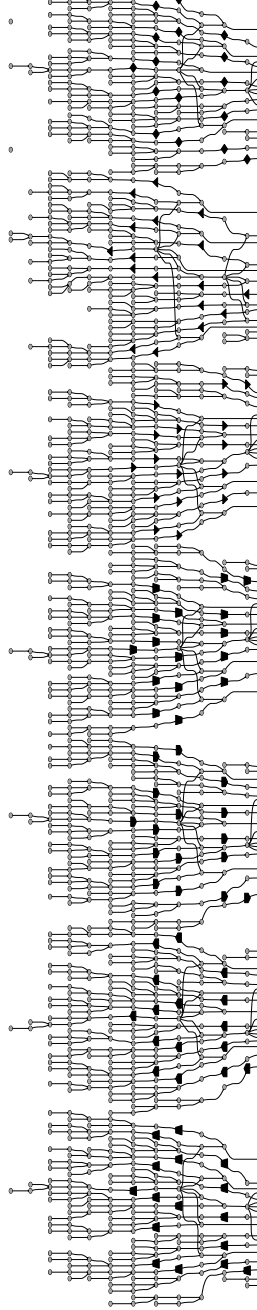


Figure A-5: Vpent

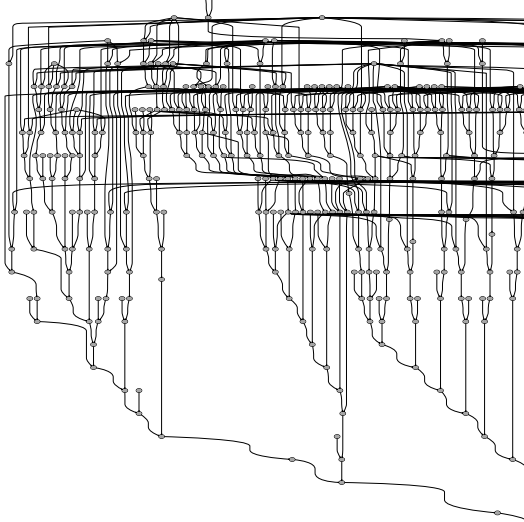


Figure A-6: Fpp

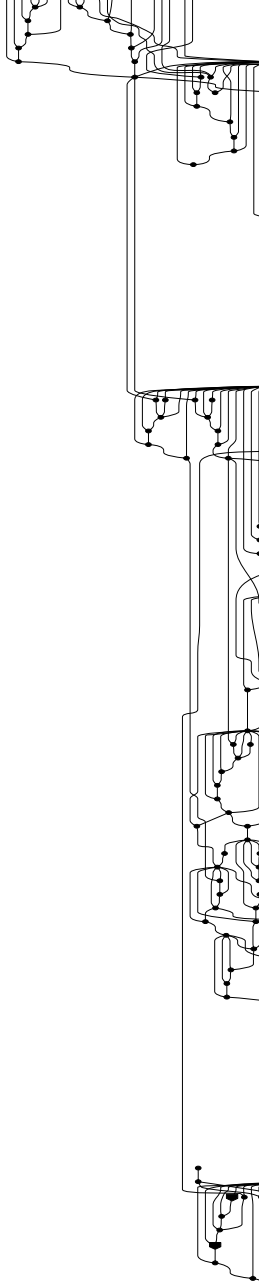


Figure A-7: Adaptive



Figure A-8: SH

- [1] V. Agarwal, M. S. Hrishikesh, S. Keckler, and M. J. H. Heule. The End of the Road for Conventional Memory Hierarchies. In *27th International Symposium on Computer Architecture*, pages 259–268, June 2000.
- [2] I. Ahmad, Y. Kwok, and M. Wu. An Efficient Scheduling Algorithm for Scheduling Task Graphs on Multiprocessors. In *the Second International Symposium on High Performance Networks*, pages 207–213, June 1996.
- [3] J. Babb, M. Frank, V. Lee, E. Waingold, S. Haktuni, and A. Agarwal. The Raw Benchmark Suite for General Purpose Computing. In *IEEE Symposium on Custom Computing Machines*, Napa Valley, CA, 1999.
- [4] R. Barua, W. Lee, S. Amarasinghe, and A. Agarwal. A Memory System for Raw Machines. In *International Symposium on Computer Architecture*, pages 199–208, 1999.
- [5] Rajeev Barua. *Maps: A Compiler-Motivated Memory-Exposed Architecture*. PhD dissertation, University of California, Berkeley, January 2000.
- [6] Standard Performance Evaluation Corporation. <http://www.spec.org/>.

- [9] Emden R. Gansner, Eleftherios Koutsoufakis, and David S. Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, March 1993.
- [10] Samuel Larsen, Emmett Witchel, and David S. Vo. Increasing and Detecting Memory Alignment. MIT/LCS, November 2001.
- [11] C. Lee, M. Potkonjak, and W. H. Mangione-Smith. Synthesizing Multimedia and Communication Scheduling. *30th Annual International Symposium on Computer-Aided Design*, Research Triangle Park, NC, December 1999.
- [12] W. Lee, R. Barua, M. Frank, D. Srikrisna, and S. Rasinghe. Space-Time Scheduling of Instruction Streams on a VLIW Machine. In *Proceedings of the Eighth International Conference on VLSI Design and Support of Programming Language and Tools*, pages 10–19, 1998.
- [13] P. Lowney, S. Freudenberger, T. Karzes, and J. Ruttenberg. The Multiflow Trace-Driven Approach to Parallel Computing. *ACM Computing Surveys*, 19(1):51–142, January 1986.
- [14] R. Nagarajan, K. Sankaralingam, D. Bu, and David S. Vo. Evaluation of Grid Processor Architectures. In *Proceedings of the Symposium on Microarchitecture (MICRO-32)*, pages 10–19, 1999.

bridge, Massachusetts, 1989.

- [17] Bruce Schneier. *Applied Cryptography: A Handbook of Algorithms, Techniques, and Protocols in C*. John Wiley, New York, second edition, 1996.
- [18] M. D. Smith. Extending SUIF for Machine Architecture. *Proceedings of the First SUIF Compiler Workshop*, 1996.
- [19] E. Waingold, M. Taylor, V. Sarkar, W. L. Hung, S. Devabhaktuni, R. Barua, J. Babb, S. Edelkamp, and R. L. Wright. It All To Software: Raw Machines. *Communications of the ACM*, 39(12), December 1996.
- [20] R. Wilson, R. French, C. Wilson, S. A. Chong, S. Liao, C.-W. Tseng, M. Hall, M. Lam, and J. R. Larusson. A Framework for Research on Parallelizing and Compiling. *ACM Notices*, 29(12), December 1996.
- [21] T. Yang and A. Gerasoulis. List Scheduling for Parallel Computing. *Parallel Computing Journal*, 19:1321–1340, 1993.